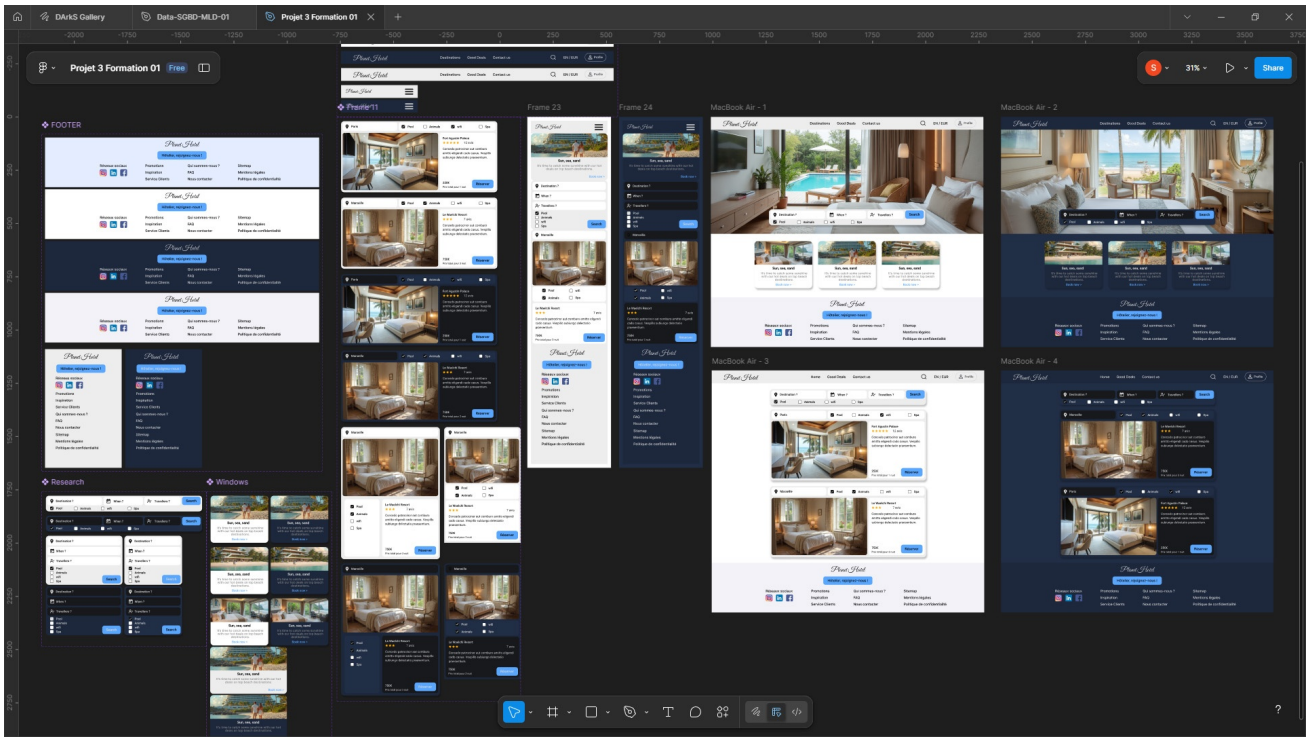
Site de Réservation d'hôtels

Par Serge Da Silva Dias

1. Introduction & Contexte :
2. Interface Web, Ergonomie et Accessibilité (Bloc 1 - HTML/CSS/JavaScript)
3. Conception et Modélisation des Données
4. Architecture Backend et Sécurité (Bloc 2 - Node.JS)
5. Développement de la Single Page Application sécurisée (Bloc 3 - framework Angular)
6. Tests et Validations
7. Compléments (SEO, Git, RGPD, Tailwind)
8. Guide utilisateur
9. Conclusion

1. Introduction & Contexte :

- **Présentation du contexte**
    - **Projet choisi :** Développer une application web permettant aux utilisateurs de réserver des services. L'application doit offrir une expérience utilisateur optimale sur des appareils de différentes tailles (smartphones, tablettes, ordinateurs de bureau) et respecter les principes d'accessibilité web.
    - Ce projet me permettait de traiter des problématiques métiers complexes :
        - **Gestion de comptes :** Admin / Client → CRUD (profile, liste d'achats, ...)
        - **Payement en ligne** (type eCommerce) sécuriser → Stripe
        - **Sécurité :** empêcher les accès non autorisé → Session / RBAC ou JWT / cryptage
- L'approche "Hybride" de l'architecture.
    - J'ai commencé par le design UI/UX sous Figma mais j'avais beaucoup d'interrogation sans réponse sur les fonctionnalités (par exemple : quels champs mettre pour les formulaires)



- Pour répondre à ces interrogations, j'ai décide de traiter le **Backend** (Bloc 2) en premier.

- J’ai ainsi fait du "**Back-end First Development"**
    - **Clarté :** Permet de définir d’abord les règles métier, les APIs, et la structure des données. Cela permet d’avoir une base solide et cohérente avant de concevoir l’interface.
    - **Optimisation :** Permet de se concentrer sur l’optimisation des requêtes, la gestion des données et la sécurité (authentification, autorisation, etc.) dès le début, sans être distrait par les aspects visuels.
    - **Stabilité** : Le back-end est souvent plus complexe et nécessite une réflexion approfondie sur les **données**, la **sécurité** et la **logique métier**.

- **Choix technologiques : Node.js/Angular**
    - **Bloc 1 :** HTML5 + CSS3 + JavaScript
    - **Bloc 2 :** Node.js (22.21.0) + Base de données : MySQL + Express (Serveur HTTP, routage)

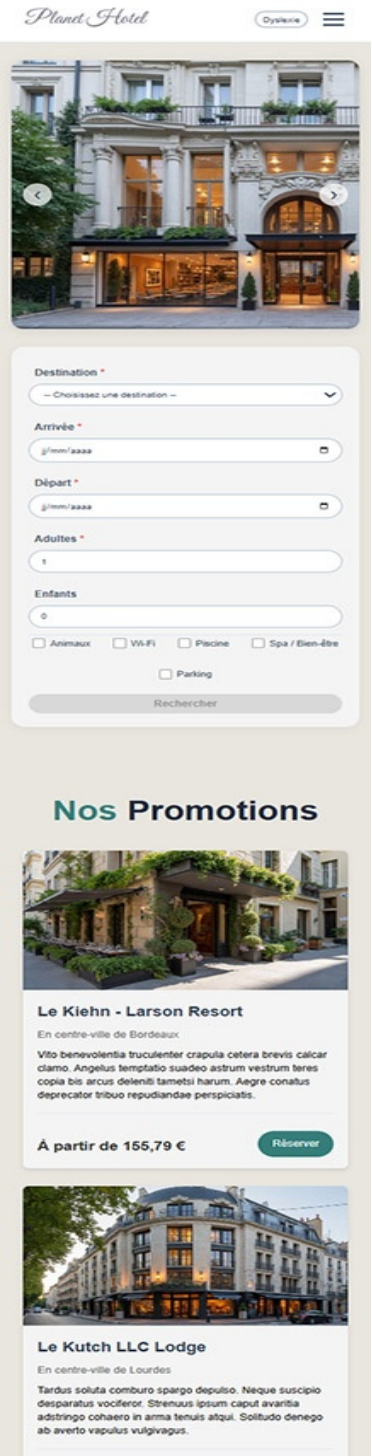
      Extensions et Librairies : dotenv (Sécurité de l'environnement) + mysql2 (Interaction BDD) + Stripe (Paiement sécurisé) + Faker.js (Environnement de test) + nodemon (Confort de développement) + Bcrypt (Algorithme de chiffrement)
    - **Bloc 3 :** Angular (20.3.0)
        - **Structure et Maintenabilité :** Architecture claire (composants, services, modules), ce qui facilite la maintenance et l’évolutivité des projets, surtout pour les grandes applications ou les équipes nombreuses.
        - **TypeScript :** ajoute un typage statique. Cela réduit les erreurs à la compilation et améliore la qualité du code, surtout pour les projets complexes.
        - **Écosystème complet :** Angular offre une solution tout-en-un : routage, gestion d’état (avec RxJS), formulaires réactifs, tests intégrés, etc. Moins besoin de bibliothèques externes qu’avec React ou Vue.

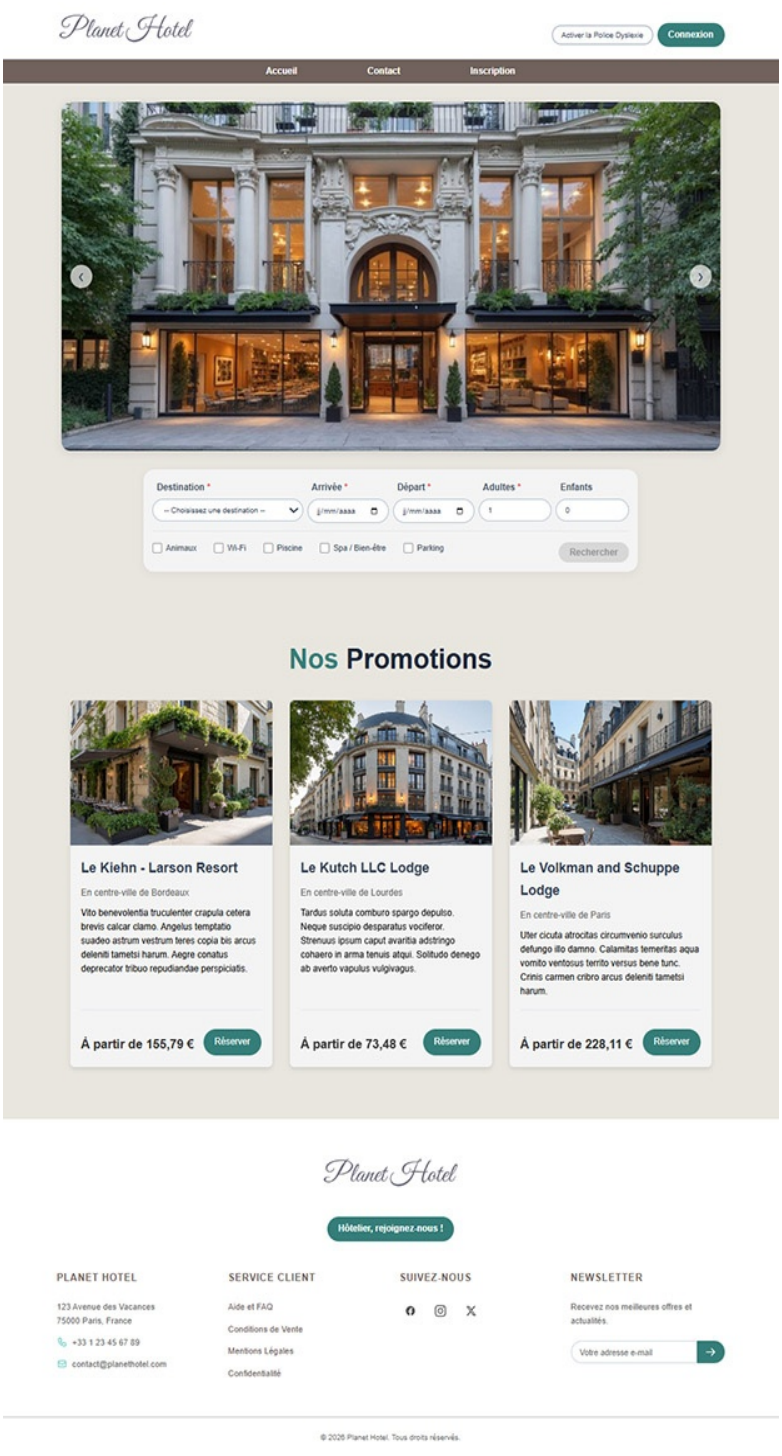
    - C’est proche d’une **Stack MEAN :** idéal pour création d’applications modulaire et maintenable à grande échelle

2. Interface Web, Ergonomie et Accessibilité (Bloc 1 - HTML/CSS/JavaScript)

2.1. Approche UI/UX et Évolution du Design : L'interface utilisateur a fait l'objet d'une réflexion à plusieurs étapes du développement afin de travailler l'ergonomie.

- **Environnement Serveur (Bloc 1) :** Le point de départ a été un template générique orienté "voyage global". Il a nécessité une refonte structurelle pour l'adapter sur le métier de la réservation d'hôtels (modification des champs de recherche, adaptation des cartes de résultats).





- **L'Itération de Conception :** L'implémentation finale diffère volontairement des maquettes et templates initiaux. Le formulaire de recherche, par exemple, a été repensé. L'ajout de champs explicites ("Nombre d'enfants", "Date de départ" dissociée) a permis de réduire la charge cognitive de l'utilisateur, privilégiant ainsi l'utilisabilité à l'esthétique pure.

2.2. Stratégie Responsive (Compatibilité multi-écrans) L'application adopte une approche "Mobile-First" pour garantir une expérience fluide sur tous les terminaux.

- **Technologies de rendu :** L'agencement global repose sur l'utilisation combinée de **CSS Grid** (idéal pour le formulaire de recherche) et de **Flexbox** (pour les alignements comme les barres de navigation et les cartes).
- **Media Queries :** Des points de rupture (breakpoints) stratégiques ont été définis pour restructurer dynamiquement l'interface (passage d'un affichage en colonnes sur desktop à un affichage empilé sur mobile).
- **Tailwind** est de base un framework orienté "Mobile-First"

2.3. Mise en conformité Accessibilité (WCAG / A11y) L'accessibilité web fait partie des contraintes architecturales importantes des sites, pour répondre aux normes d'inclusion numérique.

- **Contrastes et Lisibilité (WCAG) :** Les palettes de couleurs (du template HTML/CSS comme de la maquette Figma) ont été altérées en production. Les couleurs de fond et d'accentuation ont été assombries ou éclaircies afin de valider les ratios de contraste de luminance exigés par les normes WCAG (niveau AA/AAA). La taille typographique de base a également été augmentée.
- **Sémantique et Lecteurs d'écran (ARIA) :** L'arborescence HTML respecte strictement les balises sémantiques (<main>, <nav>, <section>).
- **Navigation au clavier :** Le parcours utilisateur (de la recherche à la réservation) a été testé et validé pour une utilisation exclusive au clavier (gestion du focus via la touche Tab et validation via Entrée), indispensable pour les utilisateurs souffrant de handicaps moteurs.
- **Inclusion Cognitive :** Intégration d'un module spécifique permettant d'activer à la volée la police typographique **OpenDyslexic**. Ce script modifie dynamiquement le DOM pour soulager l'effort de lecture des utilisateurs dyslexiques, avec persistance du choix via le navigateur.

3. Conception et Modélisation des Données

3.1 Analyse des besoins et cas d'utilisation

- Cas 1 : Le Visiteur (Utilisateur non connecté)

Le visiteur a des droits très limités. Son but est de découvrir l'offre et de devenir client.

- **Cas d'utilisation :** Rechercher un hôtel.
    - Filtrer la recherche (par ville, dates, capacités, ...).
- **Cas d'utilisation :** Consulter la fiche détaillée d'un hôtel et ses chambres.
- **Cas d'utilisation :** Créer un compte utilisateur (S'inscrire).

- Cas 2 : Le Client (Utilisateur connecté)

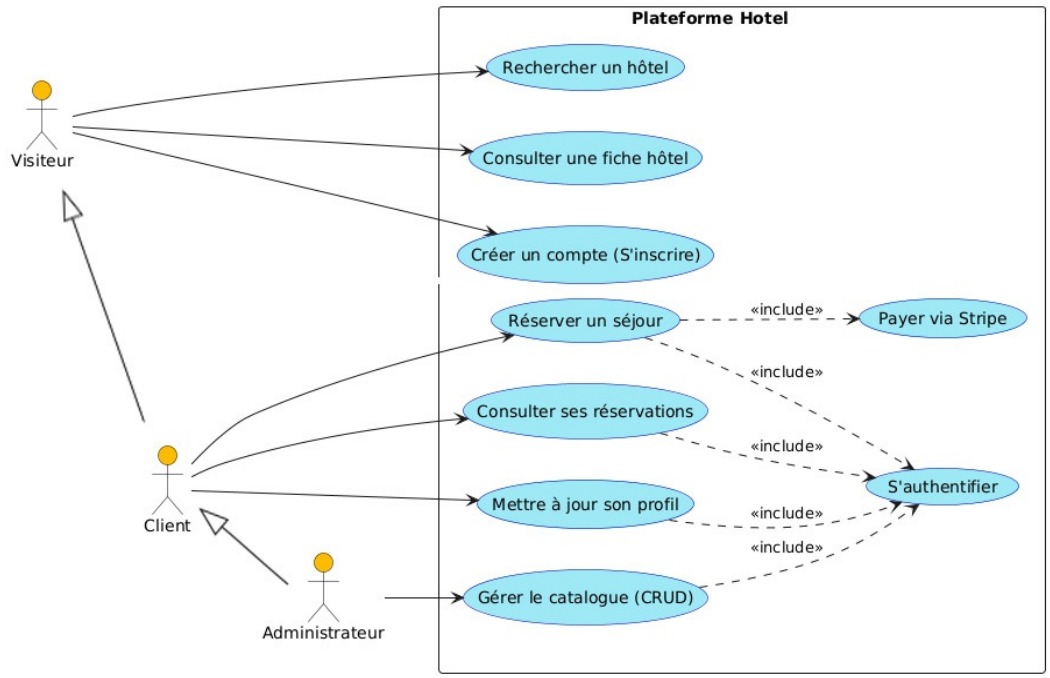
Note UML : Le Client "hérite" du Visiteur. Il peut faire tout ce que fait le visiteur, plus ses propres actions.

- **Cas d'utilisation principale :** Réserver un séjour.
    - S'authentifier. *(Obligatoire pour réserver)*
    - Payer via Stripe. *(La réservation n'est validée que si le paiement Stripe aboutit).*
- **Cas d'utilisation :** Consulter son historique de réservations.
- **Cas d'utilisation :** Annuler une commande.
    - S'authentifier.
- **Cas d'utilisation :** Mettre à jour ses informations personnelles (mot de passe, adresse, téléphone, ...).

- Cas 3 : L'Administrateur (ou Prestataire)

C'est le gestionnaire du système. Son interface (dashboard CRUD) est totalement différente.

- **Cas d'utilisation :** Gérer le catalogue des hôtels (Ajouter, Modifier, Supprimer).
    - S'authentifier en tant qu'Admin (Vérification du rôle via Guard/Middleware).
- **Cas d'utilisation :** Gérer les images associées aux hôtels.
- **Cas d'utilisation :** Gérer les chambres (Définir les prix de base, la capacité).



3.2 Structure Globale et Objectifs

Ma Base de données est conçue pour gérer un système de réservation d'hôtels en ligne, avec :

- **Gestion des utilisateurs** (clients, prestataires, admins)
- **Gestion des hôtels et de leurs chambres**
- **Réservations et commandes**
- **Avis et évaluations**
- **Gestion des images** (pour hôtels et chambres)
- **Formulaire de contact**

3.3 Les tables

clients : Stocke les informations personnelles des clients (nom, prénom, coordonnées).

- **Points importants** :
  - email UNIQUE évite les doublons.
  - Adresse complète (pays, ville, code postal, rue) pour la facturation/livraison.
- **Lien** : Liée à connexions via client_id.

Connexions : Gère l'authentification et les rôles des utilisateurs.

- **Points importants** :
  - mot_de_passe_hash pour la sécurité.
  - role permet de différencier clients, prestataires et admins.
  - client_id UNIQUE : un client = un seul compte.
- **Lien** : user_id dans hotel référence cette table pour identifier le propriétaire de l'hôtel.

hotel : Stocke les informations générales sur chaque hôtel.

- **Points importants** :
  - Champs booléens pour les équipements (piscine, spa, etc.).
  - user_id lie l'hôtel à un utilisateur (prestataire/admin).
  - currency pour gérer les devises.
- **Lien** : Liée à hotel_images, chambres, commandes, avis.

hotel_images : Gère les images associées à un hôtel.

- tag pour catégoriser les images (ex : "façade", "piscine", "réception").
- ON DELETE CASCADE : suppression automatique des images si l'hôtel est supprimé.

Chambres : Décrit les types de chambres disponibles dans un hôtel.

- prix_base, prix_enfant_sup, reduction_pourcentage pour une tarification flexible.
- date_fin_promo pour gérer les promotions temporaires.

- **Lien** : Liée à chambre_images et lignes_commande.

chambre_images : Stocke les images des chambres.

- Même logique que hotel_images pour la gestion des médias.

Commandes : Enregistre les réservations effectuées par les clients.

- date_sejour_debut et date_sejour_fin pour la gestion des disponibilités.
- statut_commande pour suivre l'état (confirmée, annulée, etc.).
- montant_total pour la facturation.
- **Lien** : Liée à lignes_commande pour le détail des chambres réservées.

lignes_commande : Détaille les chambres réservées dans une commande.

- quantite, nbr_nuits, prix_total_ligne pour un calcul précis du coût.
- prix_unitaire_facture permet de conserver le prix au moment de la réservation (même si le prix de la chambre change ensuite).

Avis : Permet aux clients de noter et commenter les hôtels.

- note avec contrainte CHECK pour une note entre 0 et 5.
- ON DELETE SET NULL pour client_id : conserve l'avis même si le client est supprimé.

Contact : Gère les messages envoyés via un formulaire de contact.

- rgpd pour la conformité légale.
- statut pour suivre le traitement des messages.

3.4 Schéma Relationnel Simplifié

clients ←→ connexions ←→ hotel

hotel ←→ hotel_images

hotel ←→ chambres ←→ chambre_images

hotel ←→ commandes ←→ lignes_commande

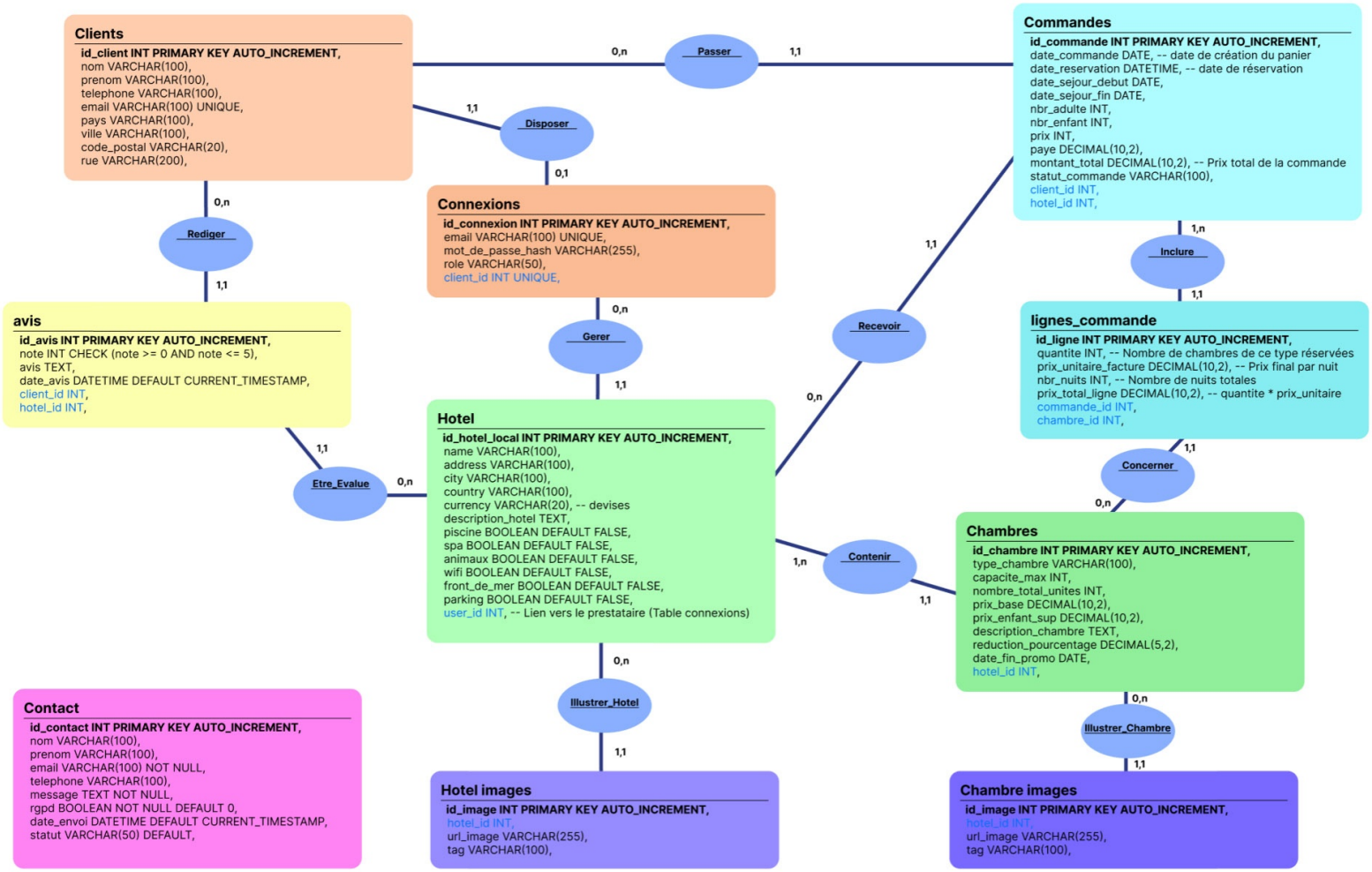
hotel ←→ avis

contact (indépendant)

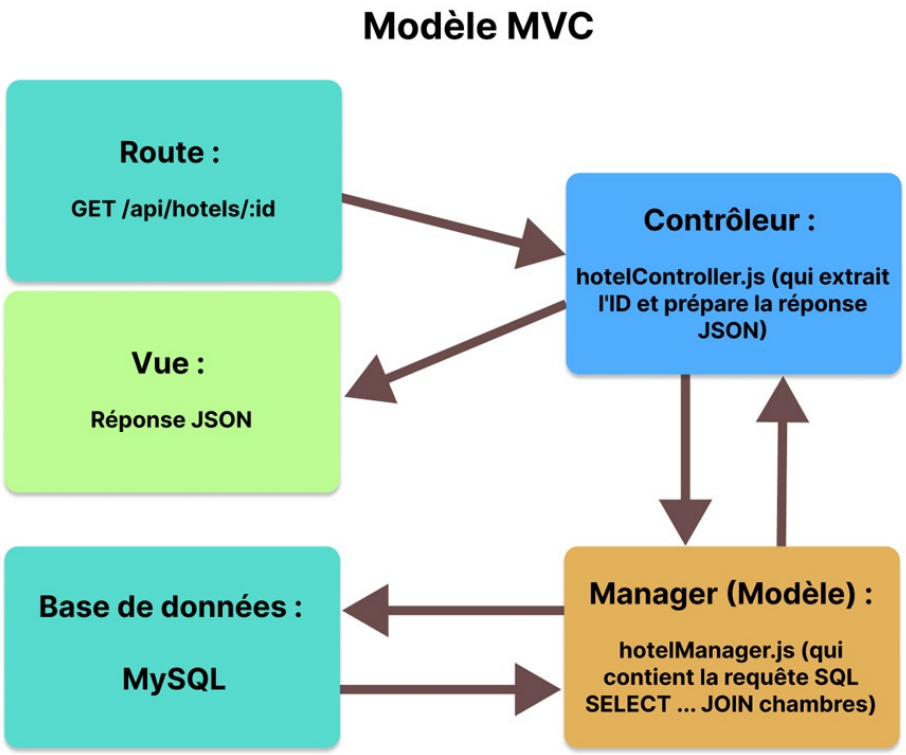
3.5 générateur de base de données Fake (Seed.js)

seed.js est un script autonome qui peut être lancé indépendamment du serveur (node seed.js) pour remplir la base de données avec des données fictives réalistes, grâce à la librairie Faker.js ou bien en utilisant des « array » pour limiter les possibilités (ex : les villes, les devises, ...).

3.6 Modélisation des Données (MCD)



4. Architecture Backend et Sécurité (Bloc 2 - Node.JS)



4.1. Le Design Pattern MVC et l'Architecture Hybride

- Le serveur **Node.js** agit comme un chef d'orchestre qui alimente le front Angular tout en gérant la logique métier.
- **Structure API REST (Client Riche - Angular) :**
  - **Serveur (server.js) :** Point d'entrée, configure les middlewares (app.use) et centralise les routes.
  - **Routes :** Répartissent le trafic (hotelRoutes, authRoutes) vers les bons contrôleurs.
  - **Controllers API (Back) :** Reçoivent les requêtes du Front, interrogent les Managers, et renvoient les données au format brut (*res.json()*).
  - **Managers (Modèles) :** Exécutent les requêtes SQL.

4.2. Authentification et Autorisation (RBAC)

- **Mécanisme d'Inscription :** Création d'un compte unique (Email/Mot de passe).
- **Sécurisation des mots de passe :** Utilisation de la librairie bcrypt. Le mot de passe n'est jamais stocké en clair. Un "salt" (grain de sel / niveau de complexité) est généré avant le hachage (*await bcrypt.hash(password, salt)*).
- **Processus de Connexion (Login) :** Recherche de l'utilisateur par email via le UserManager. Comparaison sécurisée du mot de passe saisi avec le hash en base (*await bcrypt.compare*). Si le test réussit (match = true), la session est ouverte.

4.3. Gestion des Sessions et de l'État

- **Le principe de la Session :** Mécanisme stockant les informations de l'utilisateur en mémoire côté serveur, lié au client via un cookie sécurisé contenant un identifiant unique (Session ID). Il a un contrôle centralisé et une sécurité renforcée par rapport aux JWT.
- **Les rôles de la session :**
    - Maintenir l'état de connexion.
    - Stocker les données temporaires (ID client, Rôle).
    - Personnaliser l'UI (ex: modification du header).

4.4. Sécurité et Intégrité des Données

- **Prévention des Injections SQL :** Bannissement de la concaténation de chaînes. Utilisation exclusive de **requêtes préparées** (exécuté avec des paramètres '?') pour neutraliser le code malveillant (injection SQL).
- **Protection des Routes (Middlewares & Guards) :**
    - Exécution de scripts de contrôle avant d'atteindre la logique métier (authMiddleware, adminMiddleware).
    - Vérification des permissions selon le rôle (RBAC - Role-Based Access Control).
    - Le middleware bloque (401 Unauthorized ou 403 Forbidden) ou redirige (*res.redirect*) selon le contexte d'appel (API).

- **Intégrité des Données (Transactions SQL) :** C'est une protection vitale pour la cohérence des données. Utilisation des commandes *beginTransaction*, commit (si succès complet), et rollback (annulation en cas d'erreur dans le bloc catch) pour éviter les enregistrements partiels en base de données.
- **Performance et Stabilité (Pool de Connexion) :** Utilisation de mysql.createPool (limité à 10 connexions simultanées via .env) pour éviter la surcharge du serveur de base de données en cas de fort trafic.

4.5. Intégration de l'API Tiers : Le Flux de Paiement Stripe

Le tunnel de réservation se base sur l'API externe Stripe.

4.5.1. Sécurité et Configuration : L'authentification auprès de l'API Stripe s'effectue via une clé secrète stricte (*STRIPE_SECRET_KEY*), stockée exclusivement dans les variables d'environnement (*.env*) du serveur Node.js. Cette clé n'est jamais exposée au client Angular.

4.5.2. Création de la Checkout Session : Lorsqu'un client valide sa commade le serveur Node.js compile les données de la commande et interroge l'API Stripe (*stripe.checkout.sessions.create*). Stripe renvoie une URL de paiement unique et chiffrée.

4.5.3. Redirection (Front-End) : L'application (Angular) reçoit cette URL et redirige l'utilisateur vers la page hébergée par Stripe, garantissant que les données bancaires ne transitent jamais par notre propre base de données.

4.5.4. Validation de l'intégrité post-paiement : À l'issue de la transaction, l'utilisateur est redirigé vers une URL de succès (*success_url*).

- Pour éviter les fraudes (un utilisateur tapant manuellement l'URL de succès), le serveur interroge à nouveau l'API Stripe en utilisant l'ID de session pour vérifier que le statut monétaire est bien défini sur paid (*stripe.checkout.sessions.retrieve()*) avant de déclencher l'écriture de la commande définitive dans la base de données (Manager SQL).

4.6. Test Postman : pour tester strictement la partie API sans lien avec un Front-end

Méthode POST : http://localhost:3000/api/auth/register

```
{
  "nom": "ABC1",
  "prenom": "abc1",
  "email": "abc1@test.test",
  "password": "Test123++",
  "telephone": "1234567890"
}
```

Méthode POST : http://localhost:3000/api/auth/login

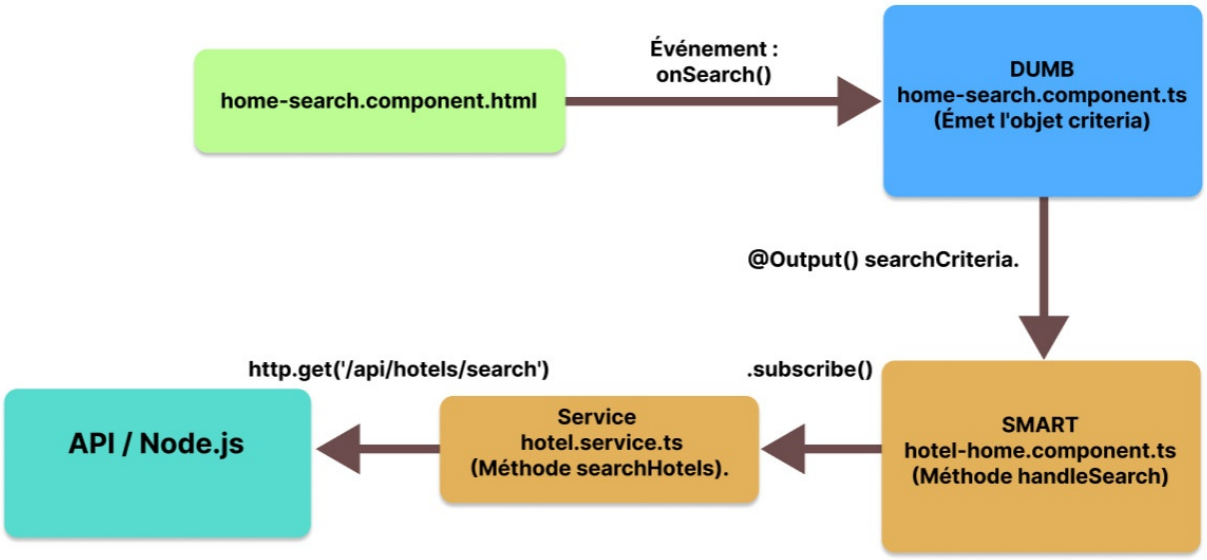
```
{
  "email": "abc1@test.test",
  "password": "Test123++"
}
```

Méthode GET : http://localhost:3000/api

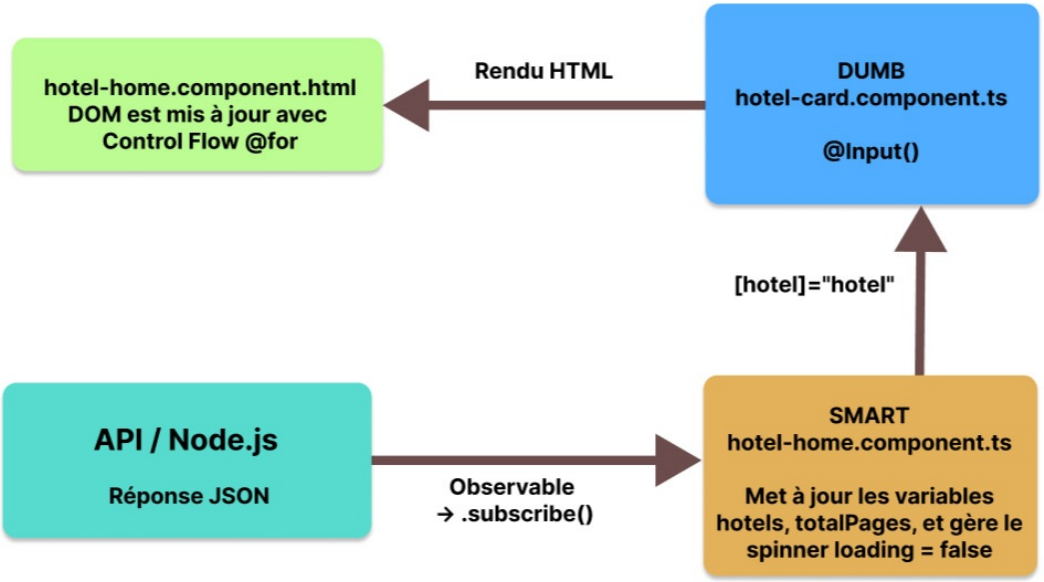
5. Développement de la Single Page Application (Bloc 3 - framework Angular)

- **SPA :** architecture modulaire, réactive, et sans rechargement de page. Il utilise un code typé (TypeScript) et hautement testable.
- **Organisation en Smart/Dumb components :**
  - **Composants "Smart"** (S'occupent de QUOI afficher et QUAND) **:** Ils sont responsables de la logique métier et de l'état de la vue. Par exemple, le HotelListComponent orchestre l'affichage des hôtels de la page d'accueil : il gère le cycle de vie (ngOnInit), injecte les services pour récupérer la liste globale des hôtels depuis l'API, et distribue cette donnée.
  - **Composants "Dumb"** (S'occupent de COMMENT afficher) **:** Ils ne s'occupent pas du reste de l'application et se concentrent uniquement sur l'interface (UI). Le HotelCardComponent se contente de recevoir un objet via un décorateur @Input() pour l'afficher. De même, le HomeSearchComponent gère la saisie utilisateur dans le formulaire réactif et remonte un événement au composant parent via un @Output(), sans jamais interroger la base de données lui-même.

Flux de données Angular : Architecture Smart / Dumb



Flux de données Angular : Architecture Smart / Dumb



- **Standalone components** : Chaque composant est autonome
- **Communication Back/Front :**
    - **Services et Appels HTTP :** Les interactions avec le backend sont centralisées dans des classes de services (ex: HotelService). Ces services utilisent le module HttpClient pour effectuer les requêtes (GET, POST, PUT, DELETE).
    - **Mapping et Typage :** Les réponses JSON brutes renvoyées par le serveur sont immédiatement mappées et validées grâce à des interfaces TypeScript strictes (Hotel, Commande), pour garantir la cohérence des données manipulées par le Front-End.
    - **Proxy de Développement :** Pour contourner les politiques de sécurité CORS (Cross-Origin Resource Sharing) lors de la phase de développement, un fichier de configuration proxy.conf.json a été mis en place. Il agit comme un intermédiaire transparent : le navigateur Angular (port 4200) effectue ses requêtes en local, et le proxy les redirige silencieusement vers le serveur Node.js (port 3000).

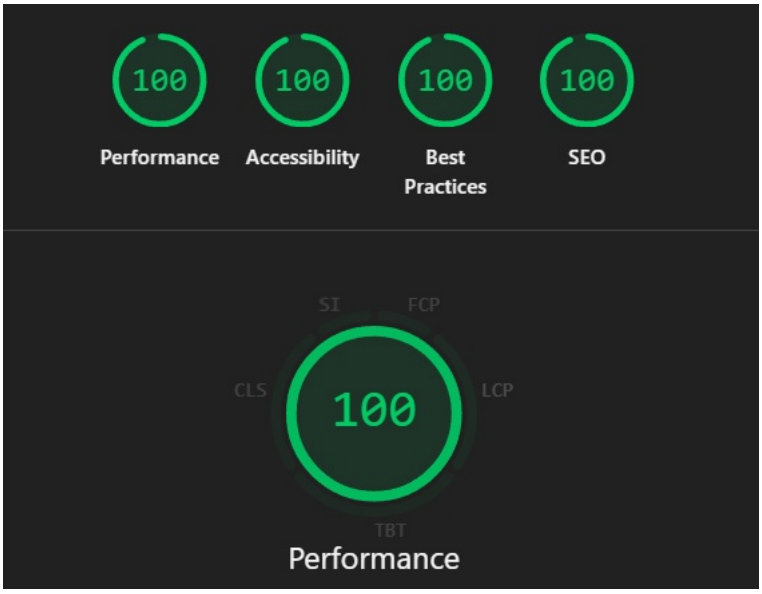
- **Routage et Protection :**
    - La récupération des données (comme *searchHotels(criteria)*) ne bloque pas l'interface. Elle retourne un flux de type *Observable<Hotel[]>*. L'exécution réelle de la requête vers la base de données n'est déclenchée que lorsqu'un composant y souscrit activement (*.subscribe()*), optimisant ainsi les ressources du navigateur.
    - Sécurisation des vues AuthGuard/AdminGuard : Le routeur Angular intègre des intercepteurs de navigation (Guards) via la fonction *CanActivateFn*.
        - **AdminGuard :** Assure une vérification stricte des autorisations RBAC (Role-Based Access Control). Il vérifie le profil de session et rejette l'accès au tableau de bord (Dashboard CRUD) si la condition user.role === 'administrateur' n'est pas strictement remplie.
        - **AuthGuard :** Bloque l'accès aux pages sensibles (comme le profil ou le tunnel de réservation) et redirige vers le login si le client n'est pas identifié.

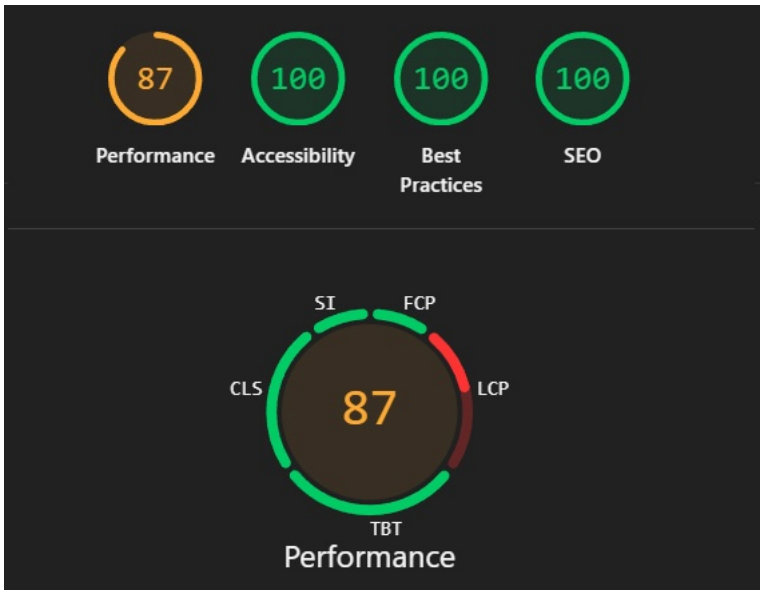
- **Angular (Stateful côté client) :** L'application garde l'utilisateur en mémoire locale.
- **Le problème du rafraîchissement (F5) en SPA :** Pour éviter l'amnésie d'Angular lors d'un F5, un Guard (admin-guard.ts via CanActivateFn) fait appel à *authService.checkAuth()*. Ce service relance silencieusement une vérification auprès de l'API pour restaurer le contexte de sécurité avant d'afficher la page.

6. Tests et Validations

Scénario de Test	Action Utilisateur	Résultat Attendu	Statut
Recherche avancée d'hôtel	Entrer des critères de recherches	Afficher les hôtels disponibles (cards) puis les chambres correspondantes (cards)	Succès
Inscription Utilisateur	Remplir le formulaire avec des données valides	Création en BDD, redirection vers login	Succès
Réservation	Sélectionner dates, valider, payer via Stripe	Commande enregistrée, page "Succès" affichée	Succès
Connexion utilisateur	Entrer l'identifiant d'un utilisateur existant	Accéder à la page profile	Succès
Annulation d'une commande	Action « annuler » dans « Mes Réservations »	Le statuts passe en « annuler »	Succès
Connexion Administrateur	Entrer l'identifiant d'un administrateur existant	Accéder à la page Admin	Succès
Ajouter un hôtel	Remplir le formulaire avec des données valides	Nouvelle entrée dans page Administration et dans la BDD	Succès
Éditer un hôtel	Changer des données d'un hôtel déjà créé	Modification dans la page Administration et de la BDD	Succès
Supprimer un hôtel	Utiliser le bouton « Supprimer » pour un hôtel existant	L'hôtel est supprimé de la page Administration et de la BDD	Succès
Ajouter un commentaires	Remplir le formulaire avec des données valides	Nouvelle entrée dans la BDD	Succès
Sécurité : Accès Admin	Tenter d'accéder à /admin sans être connecté	Accès refusé (Guard) et Redirection immédiate vers /login	Succès
Sécurité : Accès Profile	Tenter d'accéder à la page /profile sans identification	Accès refusé (Guard) et Redirection immédiate vers /login	Succès
Déconnexion utilisateur	Bouton déconnexion	Redirection à la home et suppression des accès client/admin	Succès

Type de Test	Outil Utilisé	Cible	Résultat / Score
Performance (Vitesse) Desktop (Bloc 1)	Google Lighthouse	Accueil (Desktop)	Perf: 100, Access: 100, Best Practises : 100, SEO: 100
Performance (Vitesse) Mobile (Bloc 1)	Google Lighthouse	Accueil (Mobile)	Perf: 87, Access: 100, Best Practises : 100, SEO: 100
Validation HTML	**W3C Validator**	Vues principales	0 Erreurs fatales 1 warning
Accessibilité (Clavier)	Manuel (Touche TAB)	Formulaire de réservation	Tous les champs sont atteignables





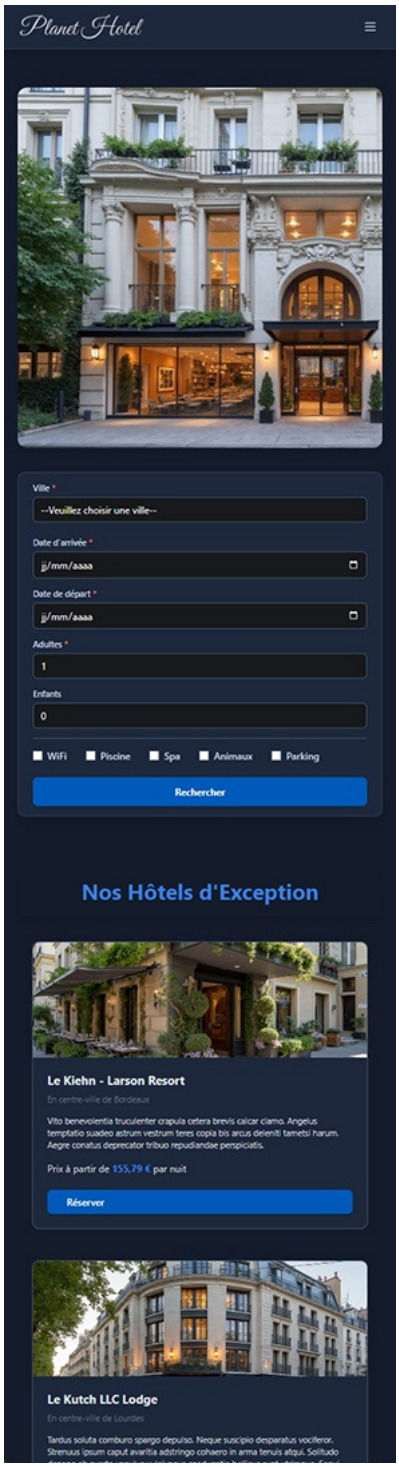
Les navigateurs de test :

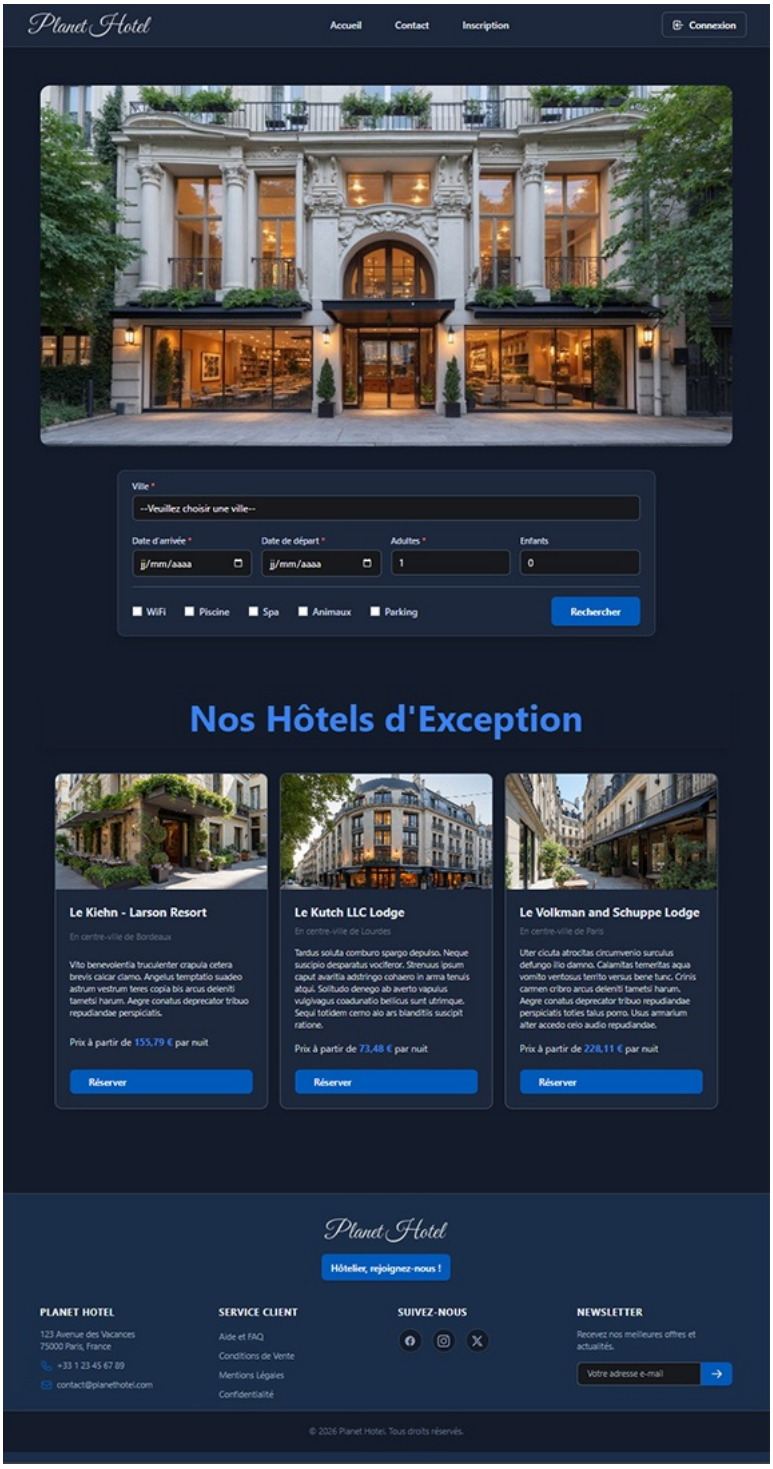
- Google Chrome
- Mozilla Firefox
- Microsoft Edge

7. Compléments

7.1. Version Angular + Tailwind

- **Environnement SPA (Bloc 3 – Angular) :** Le développement s'est basé sur une maquette Figma personnalisée → Version Dark : **Angular + Tailwind** ci-dessous





7.2. Workflow et Versionning (Git)

La gestion du code source s'appuie sur les standards de l'industrie via Git et GitHub pour assurer un suivi rigoureux :

- **Approche par Branches (Branching) :** Chaque fonctionnalité majeure (ex: feat/angular-admin-crud, feat/angular-auth) a été développée et testée de manière isolée avant d'être fusionnée.
- **Traçabilité :** Utilisation de la norme *Conventional Commits* (feat:, fix:, docs:) pour générer un historique de modifications clair et professionnel.
- **Sécurité du code :** Cette méthode garantit la réversibilité du projet à tout instant en cas de régression technique.

7.3. Protection des données sensibles et conformité RGPD

La conception applicative intègre les principes du *Privacy by Design*, garantissant la sécurité des données utilisateurs sans collecter d'informations superflues :

- **Hachage cryptographique :** Les mots de passe ne sont jamais stockés en clair. Ils sont sécurisés via l'algorithme **Bcrypt** avec l'ajout d'un "grain de sel" (salt) avant insertion en base de données.
- **Minimisation des données bancaires : Aucune coordonnée bancaire n'est stockée** sur nos serveurs. Ce traitement critique est intégralement délégué à Stripe.
- **Consentement explicite :** Le formulaire de contact intègre une validation obligatoire concernant la politique de confidentialité (RGPD) avant tout traitement de la demande en base.

7.4. Optimisation SEO (Search Engine Optimization)

Pour la partie générée côté serveur (Bloc 1), une stratégie de référencement naturel technique a été mise en place pour maximiser la visibilité sur les moteurs de recherche :

- **Sémantique HTML5 :** Structuration du contenu (<header>, <main>, <article>) et respect de la hiérarchie des titres (un seul <h1> par vue).
- **Méta-données dynamiques :** Le front Angular injecte dynamiquement des balises comme [alt]="'Photo de l\'hôtel ' + hotel.name" ou {{ hotel.description_hotel }} en fonction de l'hôtel consulté, évitant ainsi la pénalité de "Duplicate Content".
- **Balise meta description :** C’est cette description qui apparaît dans les résultats de recherche (snippet Google).
- **Attribut alt pour les images :** Les moteurs de recherche (Google, etc.) utilisent le alt pour comprendre le contenu de l’image et améliorer le référencement.
- **Performances (Lighthouse) :** L'optimisation du poids des pages et des temps de réponse du serveur Node.js contribue directement à un meilleur positionnement (Core Web Vitals).
- vérification du code HTML via **le validateur du W3C** pour avoir une indexation correcte. Les moteurs de recherche privilégient les sites bien structurés et sans erreurs.

8. Guide utilisateur

- **Navigation :** Permet d'accéder à la Home, au formulaire de contact, au formulaire d'inscription et formulaire de connexion.
- **Home :** Permet d'accéder au formulaire de recherche et aux promotions
- **Cycle de reservation :**
    - **Recherche :** Choisir une ville, une date d'arrivé à l'hôtel, une date de départ, un nombre d'adultes, un nombre d'enfants (optionnel) et des options (optionnel). Puis valider avec le bouton « rechercher »
    - Après avoir valider la rechercher un choix d'hôtels est proposer correspondant aux critères choisis. Choisir un hôtel grâce au bouton « Reserver ».
    - Après avoir choisi un hôtel, une ou plusieurs chambres disponibles sont proposés. Choisir une chambre avec « Reserver »
    - Après avoir choisi la chambre un récapitulatif est proposé. Valider et aller à la page de paiement avec « Payer et Réserver ».
    - Sur la page de paiement entrer les données bancaire et payer. Puis vous accéderez à une page de confirmation, et vous pourrez voir vos achat dans la page Profile
    - Vous pouvez annuler un réservation sur la page Profile
- **Connexion/Déconnexion :** Permet d'accéder à la page Profile (client) et/ou la Page Admin (administrateur)
- **Page Profile :** Permet de gérer ses réservations et ses données personnels.
- **Page Profile :** Permet d'ajouter/éditer/supprimer des hôtels.
- **Contact :** Permet d'envoyer un message à l'administration (ne nécessite pas d'être client). Nécessite un consentement pour le RGPD.

9. Conclusion

Pour conclure, ce projet m'a permis de traverser l'ensemble du cycle de vie d'une application Full-stack : de la conception UML à la modélisation des données, jusqu'à l'intégration front-end native et l'utilisation d'un framework complexe.

Le code a été versionné rigoureusement via Git, et l'application répond aux exigences de performance et d'accessibilité du web moderne.